

Object Oriented Pattern and Design (Java)
Course Code: ICT 2207
OOP Basics — Assignment Answers
Mawlana Bhashani Science and Technology University (MBSTU)
Department of Information and Communication Technology

SUBMITTED BY:
Md. Jahidul Islam
ID: IT24018 ; Session: 2023-24
Year: Second Semester: Second
Department of ICT, MBSTU

25 June 2026

Resumo

This document presents the answers to the five long-answer/practical questions (Section C) of the ICT 2207 — Object Oriented Pattern Design (Java) assessment booklet. The topics covered include static members and constructor overloading/chaining, abstract class vs. interface, method overriding vs. overloading, the Object class and `toString()`, and polymorphism with dynamic binding. Each answer combines a conceptual explanation with a working Java code example.

Sumário

1	Q1. Static Members & Constructor Overloading/Chaining (Understand + Apply)	2
2	Q2. Abstract Class vs. Interface (Understand + Apply)	4
3	Q3. Overriding vs. Overloading (Analyze)	6
4	Q4. The Object Class and <code>toString()</code> (Understand + Apply)	7
5	Q5. Polymorphism & Dynamic Binding (Analyze + Evaluate)	9

1 Q1. Static Members & Constructor Overloading/- Chaining (Understand + Apply)

(a) Static Variable vs. Instance Variable

A **static variable** (also called a class variable) is shared among all instances of a class. It is allocated once in the method area of the JVM when the class is loaded, and its scope is the entire class. Any change made by one object is visible to all other objects.

An **instance variable**, in contrast, belongs to an individual object. A separate copy is created in the heap for every new object, and its scope is limited to that particular object.

Why a static method cannot directly access an instance variable:

A static method belongs to the class, not to any particular object, so it can be called without creating an instance (e.g., `ClassName.method()`). At the time of the call there may be zero, one, or many objects; there is no implicit `this` reference. Because instance variables only exist inside an object, a static method has no way of knowing *which* object's variable to access, and the compiler therefore disallows the direct access.

(b) Employee Class with Constructor Chaining

```
1 public class Employee {
2
3     // Instance fields
4     private int id;
5     private String name;
6     private double salary;
7
8     // Static variable shared by ALL Employee objects
9     private static int count = 0;
10
11    // Constructor
12    public Employee() {
13        this("Unknown", 0.0);    // delegate to Employee(String
14        , double)
15    }
16
17    // Constructor
18    public Employee(String name) {
19        this(name, 30000.0);    // delegate with default
20        salary
21    }
22
23    // Constructor
24    public Employee(String name, double salary) {
25        this.id = ++count;    // count incremented EXACTLY
26        ONCE per object
27        this.name = name;
```

```

25         this.salary = salary;
26     }
27
28     // Static factory method to retrieve count
29     public static int getCount() {
30         return count;
31     }
32
33     @Override
34     public String toString() {
35         return "Employee{id=" + id + ", name='" + name +
36             "' , salary=" + salary + "}";
37     }
38
39     // Test
40     public static void main(String[] args) {
41         Employee e1 = new Employee();
42         Employee e2 = new Employee("Alice");
43         Employee e3 = new Employee("Bob", 55000.0);
44
45         System.out.println(e1);
46         System.out.println(e2);
47         System.out.println(e3);
48         System.out.println("Total Employees: " + Employee.
49             getCount());
50     }

```

Listing 1: Employee.java — static counter with chained constructors

Expected Output:

```

Employee{id=1, name='Unknown', salary=0.0}
Employee{id=2, name='Alice', salary=50000.0}
Employee{id=3, name='Bob', salary=85000.0}
Total Employees: 4

```

Key points:

- `this(...)` on the first line of a constructor delegates to another constructor in the same class.
- The increment `++count` appears only in the master constructor (Constructor 3), so it runs exactly once per object regardless of which constructor the caller uses.
- `getCount()` is `static` because it only reads a static variable and needs no object reference.

2 Q2. Abstract Class vs. Interface (Understand + Apply)

(a) Three Differences and Design Scenarios

Feature	Abstract Class	Interface
Constructors	Can have constructors	Cannot have constructors
Fields	Can have instance fields	Only public static final constants
Multiple Inheritance	A class extends only one	A class can implement many
Method Bodies	Can have concrete methods	default/static since Java 8 only

Tabela 1: Key differences between abstract class and interface

When to choose an abstract class:

If you are designing a *Vehicle* hierarchy where *Car* and *Truck* share common state (*fuelLevel*) and partially implemented behaviour (*refuel()*), use an abstract class. The shared state cannot live in an interface (no instance fields), and forcing every subclass to re-implement *refuel()* would duplicate code.

When to choose an interface:

If you want to add a *Flyable* capability to objects as diverse as *Bird*, *Drone*, and *AirCraft* (which already extend different class hierarchies), use an interface. Each class can implement *Flyable* independently while still extending its own parent, something impossible with a single abstract class due to Java's single-inheritance rule.

(b) Class/Interface Hierarchy for Car, Bicycle, and Boat

```
1 // ALL three support move()
2 interface Movable {
3     void move();
4 }
5
6 // Car and Boat share fuelLevel state and refuel() behaviour
7 abstract class MotorVehicle implements Movable {
8     protected double fuelLevel;
9
10    public void refuel(double amount) {
11        fuelLevel += amount;
12    }
13
14 }
15
16 class Car extends MotorVehicle {
17     @Override
18     public void move() { /* drive on road */ }
19 }
20
21
```

```
22 class Boat extends MotorVehicle {
23     @Override
24     public void move() { /* sail on water */ }
25 }
26
27 class Bicycle implements Movable {
28     @Override
29     public void move() { /* pedal */ }
30 }
```

Listing 2: Movable interface and MotorVehicle abstract class skeleton

3 Q3. Overriding vs. Overloading (Analyze)

(a) Four-Dimensional Comparison

Dimension	Overloading	Overriding
Method Signature	Same name, different parameters	Identical name <i>and</i> parameters
Inheritance Required	No (can occur in one class)	Yes (subclass overrides parent)
Binding Type	Compile-time (static)	Runtime (dynamic)
Single-Class Possible	Yes	No

Tabela 2: Overloading vs. Overriding across four dimensions

(b) Code Trace and Output Prediction

```
1 class Animal {
2     void sound() { System.out.println("Some sound"); }
3     void sound(int times) { System.out.println("Repeated " +
4         times + " times"); }
5 }
6 class Dog extends Animal {
7     @Override
8     void sound() { System.out.println("Bark"); }
9 }
10
11 public class Test {
12     public static void main(String[] args) {
13         Animal a = new Dog();
14         a.sound();           // Line A
15         a.sound(3);          // Line B
16     }
17 }
```

Listing 3: Animal/Dog – overloading and overriding demo

Analysis:

- **Line A** — `a.sound()`:

The compiler sees reference type `Animal` and resolves the call to `Animal.sound()` (no-arg). At runtime the actual object is a `Dog`, so dynamic binding kicks in and executes `Dog.sound()` — **overriding**.

Output: Bark

- **Line B** — `a.sound(3)`:

The compiler matches the call to `Animal.sound(int)` because `Dog` does not override the `int`-parameter version. This resolution happens purely at compile time based on the argument type — **overloading**.

Output: Repeated 3 times

Full Expected Output:

Bark

Repeated 3 times

4 Q4. The Object Class and toString() (Understand + Apply)

(a) Three Inherited Methods from Object

Every Java class implicitly extends `java.lang.Object`. Three important inherited methods are:

1. `toString()` — Returns a string representation of the object. The default implementation returns `ClassName@hashCode` (e.g., `Book@1b6d3586`).
2. `equals(Object obj)` — Tests reference equality by default (`==`), i.e., whether two references point to the same memory address.
3. `hashCode()` — Returns an integer hash code. By default it is typically derived from the object's memory address.

Default toString() format:

`getClass().getName() + '@' + Integer.toHexString(hashCode())`

Example: `Book@6d06d69c`

(b) Predicting Output and Overriding toString()

Before override: `System.out.println(b1)` implicitly calls `b1.toString()`, which uses the default `Object` implementation. The output will be something like:

`Book@<hexHashCode>` (e.g., `Book@6d06d69c`)

```
1 class Book {
2     String title;
3     String author;
4     double price;
5
6     Book(String title, String author, double price) {
7         this.title = title;
8         this.author = author;
9         this.price = price;
10    }
11
12    @Override
13    public String toString() {
14        return "Book: " + title + " by " + author + " (Price:
15            " + price + ")";
16    }
17
18    public class Test {
19        public static void main(String[] args) {
20            Book b1 = new Book("Java Basics", "Alice", 450.0);
21            System.out.println(b1);    // implicitly calls b1.
                                     toString()
```

```
22     }
23 }
```

Listing 4: Book.java with overridden toString()

Output after override:

Book: Java Basics by Alice (Price: 450.0)

Why toString() is preferred over a separate display() method:

- `System.out.println(obj)`, string concatenation (`"Info: " + obj`), and logging frameworks all call `toString()` automatically.
- A custom `display()` method requires explicit invocation and is invisible to the Java API.
- Overriding `toString()` is a standard Java convention, making the code more readable and compatible with libraries (e.g., collections print neatly with `toString()`).

5 Q5. Polymorphism & Dynamic Binding (Analyze + Evaluate)

(a) Compile-Time vs. Runtime Polymorphism

Compile-time (static) polymorphism is resolved by the compiler based on the method signature. It is achieved through *method overloading*. Example:

```
1 class MathUtils {
2     int add(int a, int b)          { return a + b; }
3     double add(double a, double b) { return a + b; }
4 }
```

Listing 5: Compile-time polymorphism via overloading

The compiler selects the correct `add` at compile time based on argument types.

Runtime (dynamic) polymorphism is resolved at runtime based on the actual object type. It is achieved through *method overriding* and a parent-type reference. Example:

```
1 Animal a = new Dog();
2 a.sound(); // resolved at runtime -- calls Dog.sound(), not
             Animal.sound()
```

Listing 6: Runtime polymorphism via overriding

Dynamic Binding is the mechanism the JVM uses to determine *at runtime* which overridden method to execute, by looking up the actual type of the object (not the declared reference type). Runtime polymorphism relies on dynamic binding; compile-time polymorphism does not.

(b) Code Trace — Shape, Circle, Square

```
1 class Shape {
2     double area() { return 0; }
3     void describe() { System.out.println("Area: " + area()); }
4 }
5
6 class Circle extends Shape {
7     double radius = 5;
8     @Override
9     double area() { return Math.PI * radius * radius; }
10 }
11
12 class Square extends Shape {
13     double side = 4;
14     @Override
15     double area() { return side * side; }
16 }
17
18 public class Test {
19     public static void main(String[] args) {
20         Shape[] shapes = { new Circle(), new Square() };
21     }
22 }
```

```

21         for (Shape s : shapes) {
22             s.describe();
23         }
24     }
25 }

```

Listing 7: Shape hierarchy – dynamic binding trace

Step-by-step trace:

1. Iteration 1 — `s` refers to a `Circle` object

- `s.describe()` is called. At compile time, the compiler sees reference type `Shape`, so it binds to `Shape.describe()`.
- At runtime, the actual object is `Circle`, so `Shape.describe()` executes — but inside `describe()`, `area()` is called.
- The call to `area()` is also dynamically bound: the actual object is `Circle`, so `Circle.area()` executes.
- `Circle.area()` returns $\pi \times 5^2 = \pi \times 25 \approx 78.53981633974483$.
- Output: Area: 78.53981633974483

2. Iteration 2 — `s` refers to a `Square` object

- `s.describe()` calls `Shape.describe()` (compile-time reference type = `Shape`).
- Inside `describe()`, `area()` is dynamically bound to `Square.area()`.
- `Square.area()` returns $4 \times 4 = 16.0$.
- Output: Area: 16.0

Full Expected Output:

```

Area: 78.53981633974483
Area: 16.0

```

Key insight: Even though `describe()` is defined in `Shape` and is NOT overridden in `Circle` or `Square`, the call to `area()` *inside* `describe()` is still dynamically dispatched. This is because method calls in Java always use dynamic binding for virtual (non-static, non-private) methods, regardless of where the call originates.